

MIGNet GRN intra 活性、 unit-specific GRN 与纯空间 spot-local GRN Codex Plan

样本级 GRN 表达加权、unit-specific GRN、纯空间 spot
local GRN pilot

面向已更新代码的增量计划：保留 native unit 无泄露框架，解决 sample-level gene-to-gene GRN 导致 intra 特征缺少 unit 独特性的问题。

本轮方案一句话

当前代码已经有 clean_grn_cci_mix 和 native unit feature 框架。本轮不回到 lower-to-upper 投影，也不引入 GRN inter；只补两条路线：第一，在样本级 GRN 上乘 unit 表达活性，新增 grn_cci_expression；第二，在 DataFactory 层面对每个 unit 生成 unit-specific GRN，再让现有 clean GRN-CCI 网络读取。spot 级别先做 selected spot/cell 的 pure-spatial local GRN pilot，即每个被挑选的中心 spot/cell 只按原始空间坐标取 50 个邻居一起跑 GRN，输出中心 spot/cell 的局部 GRN。

目录

1	当前问题与总体决策	2
2	现有代码起点	2
3	路线一：新增 GRN-CCI expression 网络	3
3.1	数学定义	3
3.2	代码修改	3
4	路线二：DataFactory 新增 unit-specific GRN 输入	4
4.1	domain/unit 层级	4
4.2	DataFactory 输出格式	4
4.3	与 MIGNet 对接	4
5	spot 级别：selected spot/cell local GRN pilot	4
5.1	为什么不是单个 spot 一行直接推 GRN	4
5.2	“挑几个细胞一起跑”的实现方式：只保留纯空间 KNN	5
5.3	pilot 脚本	5
6	feature 相似性与活性验证	6
7	导出开关与输出结构	6
8	Codex 实施顺序	7
9	验收标准与第一轮实验矩阵	7

1 当前问题与总体决策

当前更新后的代码已经完成了一个关键修正：lower 和 upper 不再通过 overlap 投影到同一个 upper 坐标系，而是在各自的 native unit 空间中构造特征，再分别计算跨时间 P_{ij} 与 EI。这一步解决了原先最主要的信息泄露风险。

新的问题来自 GRN 输入本身。DataFactory 当前对一个 sample 生成的是 gene-to-gene GRN：

$$g \rightarrow h, \quad w_{g,h}.$$

这里 g 是 regulator gene， h 是 target gene， $w_{g,h}$ 是样本级调控边权。它没有天然携带 unit 信息。因此若 intra 只读取同一张样本级 GRN 的边权，不同 unit 的 intra block 可能高度相似；只有表达 mask 可以制造离散差异，但连续活性仍然不足。

本轮不做的事情

本轮不重新引入 GRN inter，不做 GRN 与 CCI 的乘积或加法，不把 lower 投影到 upper 坐标，也不把 spot 级别的单行表达直接当作可推断 GRN 的完整数据。所有新增工作只围绕 intra 的 unit 独特性展开。

两条路线并行，但第一轮实验分开跑		
路线	核心做法	目标问题
网络侧补强	使用样本级 GRN，但让 intra 边权乘以 unit 内 regulator 与 target 的表达活性。	表达加权能否模拟 unit-specific regulatory activity?
数据侧补强	在 DataFactory 侧对每个 domain/unit 单独推断 GRN，网络侧读取 unit-specific GRN。	真正 unit-specific GRN 是否提供更强的 intra 独特性?
spot pilot	对被挑选的中心 spot/cell 只取 50 个纯空间近邻，一起跑局部 GRN。	spot 层级是否存在可扩展且不引入 domain 信息的局部 GRN 近似方案?

2 现有代码起点

可以直接复用的代码位置

- mignet_ce/networks/clean_grn_cci_mix.py: 当前 native unit 网络基线；intra 来自 GRN，inter 来自 CCI。本轮把它作为新网络父类或参考实现。
- mignet_ce/features_native.py: 负责把 intra/inter 边表转为 native unit feature matrix。本轮复用，并补 raw feature 诊断导出。
- mignet_ce/graph/builder.py: 本轮核心修改位置，增加表达加权 intra 与 unit-specific GRN 读取。

- `mignet_ce/representations/graph_features.py`: native unit 模式下直接使用 lower/upper 自身矩阵和坐标。本轮禁止恢复 overlap projection。
- `mignet_ce/metrics.py`: EI 正常计算；native unit 下 DI/TE 不兼容时保持 NaN 或 EI-only 标记。
- DataFactory 的 GRN runner 与 GRN 推断核心：复用 sample-level GRN 推断逻辑，扩展出 unit-specific runner 与 spot local pilot。

当前 feature 输出默认开启，加 `--no-export-features` 时关闭。为了本轮诊断，建议额外暴露 raw native feature、graph edge 与 feature block summary 的导出开关。

3 路线一：新增 GRN-CCI expression 网络

3.1 数学定义

样本级 GRN 给出结构：

$$g \rightarrow h, \quad w_{g,h}.$$

新增网络让每个 unit 的表达状态调制这条边的活性：

表达加权 intra 边权

$$I_{u,g \rightarrow h}^{expr} = w_{g,h} \cdot a(x_{u,g}, x_{u,h}).$$

其中 u 是 unit, $x_{u,g}$ 与 $x_{u,h}$ 是该 unit 中 regulator 与 target 的表达, $a(\cdot)$ 是表达活性函数。第一版推荐几何平均：

$$a(x_{u,g}, x_{u,h}) = \sqrt{\tilde{x}_{u,g} \tilde{x}_{u,h}},$$

其中 $\tilde{x}$ 是经过 `log1p` 与全局缩放后的表达值。相比直接乘法，几何平均仍然要求 regulator 和 target 同时活跃，但不会过度放大高表达基因。

3.2 代码修改

1. 修改 `mignet_ce/graph/builder.py` 中的 intra 构造逻辑，给 intra builder 增加表达权重参数：none、geometric_mean、product、min。
2. 新增 GRN-CCI expression 网络文件。它复用 clean GRN-CCI mix 的 native unit 结构，区别只是 intra score 使用样本级 GRN 与表达活性的乘积。
3. 修改网络注册表与配置文件，注册新的 GRN-CCI expression network method。
4. inter 固定为 CCI only，不读 GRN inter，不做 $\text{GRN} \times \text{CCI}$ ，不做 additive。
5. 在 metadata 中记录 intra 来源、表达变换方式、表达权重函数，并标记 feature alignment 仍然是 native units。

4 路线二：DataFactory 新增 unit-specific GRN 输入

4.1 domain/unit 层级

对 k150、k40、less-than5 等 domain 层级, 一个 unit 内部通常包含多个 spot/cell。unit-specific GRN 的输入不是 domain h5ad 中聚合后的一行, 而是该 unit 内部原始 spot/cell 的表达矩阵:

$$X_u \in \mathbb{R}^{n_u \times G}.$$

这里 n_u 是 unit 内 spot/cell 数量, G 是基因数。对每个 unit 独立推断:

$$GRN_u = \{g \rightarrow h, w_{u,g,h}\}.$$

此时 intra 直接使用 $w_{u,g,h}$, 不需要再用表达活性模拟:

$$I_{u,g \rightarrow h}^{unit} = w_{u,g,h}.$$

4.2 DataFactory 输出格式

新增:

- data_factory/scripts/run_unit_grn_layer.py
- data_factory/lib/unit_grn_layer_runner.py

推荐输出一个合并表:

E1S1_domain_factory/grn_unit_specific/<layer>/<sample>/unit_grn_edges.csv

字段:

unit_id, regulator, target, weight, weight_norm, n_cells, grn_status.

同时输出 summary:

unit_id, n_cells, n_genes_used, n_regulators_used, n_edges, status, reason.

4.3 与 MIGNet 对接

在 resolver 或 layer paths 中增加可选 unit-specific GRN 路径。网络侧增加 GRN source 选择:

sample, sample_expression_weighted, unit_specific.

当选择 unit_specific 时, intra 根据 unit_id 只写入对应 unit; inter 仍然只来自 CCI。某个 unit 没有局部 GRN 时, 第一版建议 fallback 到表达加权样本级 GRN, 避免少数失败 unit 中断整批实验。

5 spot 级别: selected spot/cell local GRN pilot

5.1 为什么不是单个 spot 一行直接推 GRN

spot h5ad 的表达矩阵通常是:

$$X \in \mathbb{R}^{n_{spots} \times G}.$$

每一行是一个 spot/cell，每一列是一个 gene。单个 spot 只有一行表达：

$$x_s = (x_{s,1}, x_{s,2}, \dots, x_{s,G}).$$

这一行可以描述 gene activity，但不能估计基因之间的变化关系。GRN 推断需要多行观测来学习 gene A 的变化是否解释 gene B 的变化。因此单个 spot 不能严格从头推断 GRN。

5.2 “挑几个细胞一起跑”的实现方式：只保留纯空间 KNN

对每个被挑选的中心 spot/cell s ，构造一个局部观测集合 $N_{50}^{sp}(s)$ 。本版明确删除 domain-restricted KNN 和 expression KNN，只保留方案 A：纯空间 KNN。

为什么不使用 domain 限制邻域

domain 标签本身就是后续层级比较的一部分。如果用 domain 归属限制 spot-local GRN 的邻域，就会把层级划分信息提前写进 spot 级 GRN，后续 P_{ij} 和 EI 可能受到循环信息影响。因此第一版 pilot 不使用 domain 标签、不使用 upper/lower overlap，也不使用任何后续层级归属表。

对一个被选中的 center spot/cell s ：

1. 只用原始空间坐标计算距离，选择离 s 最近的 50 个 spot/cell 作为邻居。
2. 局部表达矩阵由中心 spot/cell 与 50 个空间近邻共同组成：

纯空间局部表达矩阵

$$X_{N_{50}^{sp}(s)} \in \mathbb{R}^{51 \times G}.$$

这里 51 行表示 1 个中心 spot/cell 加 50 个空间近邻。第一版推荐把 --k-neighbors 50 定义为“不包含中心点的 50 个邻居”，并额外用 --include-center 把中心点加入局部矩阵。如果后续希望严格保持 50 行，则应改名为 --local-size 50，避免 k 是否包含中心点产生歧义。

随后对局部表达矩阵运行 GENIE3-style GRN：

$$GRN_s^{local} = \{g \rightarrow h, w_{s,g,h}^{local}\}.$$

这个输出应称为 **center-cell local GRN** 或 **spot-centered local GRN**，而不是“单个细胞一行直接推断出的 GRN”。

5.3 pilot 脚本

新增：

`data_factory/scripts/run_spot_local_grn_pilot.py`

建议命令形态：

```
python data_factory/scripts/run_spot_local_grn_pilot.py \
  --spot-h5ad E1S1_domain_factory/spot/spot_heart_11.5.h5ad \
  --selected-units spot_001 spot_023 spot_205 \
  --neighbor-mode spatial \
```

```
--k-neighbors 50 \
--include-center \
--min-cells 30 \
--output-root EIS1_domain_factory/grn_spot_local/spot/spot_heart_11.5
```

第一版 `--neighbor-mode` 只允许 `spatial`。如果代码中保留其他枚举值，也必须默认关闭，并在 README 中标记为非第一轮实验选项。

输出：

```
grn_spot_local/<layer>/<sample>/spot_local_grn_edges.csv
```

字段包括：中心 unit、邻居 unit、regulator、target、边权、归一化边权、邻居数量、是否包含中心点、邻居选择模式和运行状态。

同时输出邻居清单，便于审计没有使用 domain 信息：

```
grn_spot_local/<layer>/<sample>/spot_local_neighbors.csv
```

字段包括：中心 unit、邻居 unit、空间距离、邻居排名、是否用于 GRN 推断。

第一版建议只做诊断，不直接并入全量 ablation。先比较 pure-spatial local GRN 与 expression-weighted GRN 的 feature 相似性，再决定是否扩展。

6 feature 相似性与活性验证

新增诊断脚本：

```
scripts/compare_grn_activity_features.py.
```

输入两套 feature，例如 `grn_cci_expression` 与 `unit_specific_clean_grn_cci_mix`，输出四类指标。

同一个 unit 的 feature cosine

$$\cos(z_u^A, z_u^B).$$

unit-unit 相似结构相关性

$$S_{uv}^A = \cos(z_u^A, z_v^A), \quad S_{uv}^B = \cos(z_u^B, z_v^B),$$

$$\rho = \text{corr}(\text{vec}(S^A), \text{vec}(S^B)).$$

同时比较 unit specificity：平均 pairwise cosine distance、跨 unit feature 方差、effective rank。最后比较 P_{ij} 相关性、行熵与 EI gain。如果表达加权 GRN 与 unit-specific/local GRN 在 feature、 P_{ij} 和 EI gain 上都接近，就可以支持：样本级 GRN 乘 unit 表达活性是 unit-specific GRN 活性的低成本近似。

7 导出开关与输出结构

建议新增或明确以下开关：

- --export-raw-native-features
- --export-graphs
- --export-feature-diagnostics

建议输出结构：

```
features/<organ>/<lower>__<upper>/
  <stage>_lower_features_raw_native.csv
  <stage>_upper_features_raw_native.csv
  <stage>_lower_features_scaled.csv
  <stage>_upper_features_scaled.csv
  feature_schema.json
network_exports/<stage>_lower_intra_edges.csv
network_exports/<stage>_lower_inter_edges.csv
diagnostics/<stage>_feature_block_summary.csv
```

其中 block summary 至少包含：stage、layer role、unit id、intra 总强度、inter 总强度、intra 非零维数、inter 非零维数和 feature norm。

8 Codex 实施顺序

1. **先加 grn_cci_expression**：修改 intra builder，新增网络文件，注册 network method，补配置与 CLI，小样本 smoke test。
2. **加 raw feature 与 graph diagnostic export**：保留 scaled feature，增加 raw native feature、edge exports、block summary。
3. **加 unit-specific GRN DataFactory runner**：按 unit 内原始 spot/cell 表达矩阵跑 GRN，输出合并的 unit_grn_edges.csv 与 summary。
4. **MIGNet 支持 unit-specific GRN 输入**：resolver 识别路径，graph builder 根据 GRN source 选择样本级、表达加权或 unit-specific intra。
5. **加 spot local GRN pilot**：支持 selected center spot/cell，只使用 pure spatial KNN，固定 50 个空间邻居，不使用 domain 标签或 overlap，输出 center-cell local GRN。
6. **加 feature similarity comparison**：比较 feature cosine、unit-unit similarity、specificity、Pij 与 EI gain。

9 验收标准与第一轮实验矩阵

验收标准

1. clean_grn_cci_mix 原有结果可以继续运行。
2. grn_cci_expression 可以在 native unit 下输出 feature、Pij 和 EI。
3. 表达加权 intra 相比当前 masked intra 具有更高的 unit 间方差或 pairwise distance。

4. unit-specific GRN runner 可以为 k150、k40、less-than5 输出 `unit_grn_edges.csv`。
5. selected spot/cell local GRN pilot 可以输出 center-cell local GRN。
6. feature similarity 脚本可以定量回答表达加权 GRN 是否近似 unit-specific/local GRN 活性。
7. native unit 模式下不恢复 lower-to-upper projection；DI/TE 不兼容时保持 NaN 或 EI-only 标记。

第一轮实验矩阵

- **clean GRN-CCI mix**: 样本级 GRN, intra 使用 GRN 边权加表达 mask, inter 使用 CCI。这是当前无泄露基线。
- **GRN-CCI expression**: 样本级 GRN, intra 使用 GRN 边权乘 unit expression activity, inter 仍使用 CCI。这是低成本模拟 unit-specific activity 的版本。
- **unit-specific clean GRN-CCI mix**: DataFactory 先输出 unit-specific GRN, intra 直接读取 unit-specific GRN 边权, inter 仍使用 CCI。这是数据侧更真实的版本。
- **spot local GRN pilot**: 只对 selected spot/cell 做纯空间中心邻域 GRN, 固定 50 个空间邻居, 先用于诊断, 不直接进入全量 ablation。

第一轮推荐跑法

先跑 `clean_grn_cci_mix` 与 `grn_cci_expression`, 因为它们不需要重跑 DataFactory; 然后对 k150/k40 选少量 organ-time sample 生成 unit-specific GRN; 最后只对几个 selected spots/cells 做 local GRN pilot。先用 feature 相似性和 EI gain 判断表达加权方法是否足够近似, 再决定是否把 unit-specific/local GRN 扩到全量实验。